

Lesson 8: Designing Algorithms For Simple Programming Problems

Generated Jul 7, 2026 1:26 PM

What We'll Learn

By the end of this lesson, we can:

- Analyze a problem statement.
- Create an algorithm that solves the problem.
- Test an algorithm using sample values.

Words We'll Use

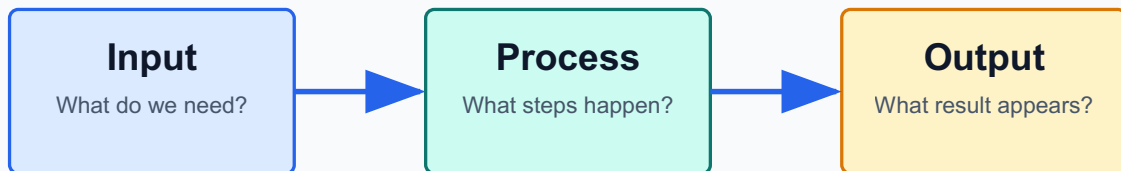
- Problem statement - description of what must be solved.
- Test data - sample values used to check.
- Logic - reasoning connecting steps.
- Trace - manually follow steps.
- Efficiency - solving without unnecessary steps.

Before We Start

How do you know if your algorithm works before coding it?

Let's Understand

Algorithm Planning: IPO



Let's work through Lesson 8: Designing Algorithms For Simple Programming Problems together. Our focus is creating algorithms independently from given problem statements. We will move from meaning, to examples, to practice, then to a task we can explain. What this lesson means:

- An algorithm is an ordered list of steps that solves a problem or completes a task.
- Before we code, we identify the input, process, and output so our solution has direction.
- Good steps are clear enough that another person can follow them without guessing.
- The order matters because computers follow instructions exactly as written.
- We can test an algorithm with sample values before writing Java code.
- A strong algorithm is specific, complete, logical, and easy to revise.

How we study it step by step:

- First, we read the problem statement and underline the exact result the program must produce.
- Next, we list the required input values before writing any solution steps.
- Then, we choose the process: compute, compare, count, convert, or decide.
- After that, we write ordered steps using action words that another student can follow.
- We test the algorithm with at least two sample inputs, including one boundary or special case.
- Finally, we revise any vague step until the expected output is clear before coding.

Rules and patterns we should remember:

- Rule: Use action words such as read, compute, compare, display, repeat, or stop. Example: Step 1: Read firstNumber. Step 2: Read secondNumber. Step 3: Compute sum. Check: Which word tells us the action in Step 3?
- Rule: Put one clear action per step when possible. Example: Better: Read grade, then Compare grade to 75; unclear: Check everything. Check: Which version can a beginner follow?

- Rule: Include the needed input before the process step. Example: For sum of two numbers, we first need number1 and number2 before computing sum. Check: What information must we ask for first?
- Rule: Include the output after the process or decision step. Example: After computing $\text{sum} = 5 + 7$, we display 12. Check: What should the user see at the end?
- Rule: Test the algorithm with at least one sample value. Example: If number1 is 5 and number2 is 7, the expected sum is 12. Check: Does our step list produce 12?

Common mistakes we will watch for:

- Writing a paragraph instead of ordered steps.
- Starting with computation before identifying inputs.
- Skipping the output step.
- Combining several actions into one vague step.
- Not testing whether another person can follow the algorithm.

Mini-check before practice:

- What is our input?
- What process happens to the input?
- What output should appear?
- Are the steps clear enough for a classmate to follow?

When we move to practice, our goal is not only to get an answer. Our goal is to explain the thinking: what information we used, what step happened next, why the result is correct, and what we changed after testing.

Let's Practice

Let's practice with these activities:

Activity 1: Problem Analysis Type: individual Time: 8 minutes Instruction: Read a problem and identify what must be solved. Student task:

- Problem: display the larger of two numbers.
- Underline the needed input.
- Write the decision question.
- Write the expected output.

Output: Submit problem notes with input, decision, and output.

Activity 2: Build The Algorithm Type: individual Time: 12 minutes Instruction: Create a complete algorithm for a decision problem. Student task:

- Start.
- Read num1 and num2.
- Compare num1 and num2.
- Display the larger number.
- End.
- Add clear Yes/No or if/else wording.

Output: Submit the complete larger-number algorithm.

Activity 3: Trace Table Type: pair Time: 10 minutes Instruction: Test the algorithm using sample values. Student task:

- Trace num1 = 8, num2 = 3.
- Trace num1 = 4, num2 = 9.
- Trace num1 = 6, num2 = 6.
- Write what should display each time.

Output: Submit three trace rows.

Activity 4: Peer Debug Type: pair Time: 10 minutes Instruction: Review a partner's algorithm for missing or unclear logic. Student task:

- Check if input is complete.
- Check if the decision is clear.
- Check if every path has output.
- Suggest one revision.

Output: Submit peer feedback and revised algorithm.

Activity 5: Exit Ticket Type: exit ticket Time: 5 minutes Instruction: Explain testing before coding. Student task:

- Why do we trace an algorithm?

- What sample values did we use?
- What did we revise?

Output: Submit three short answers.

Now We Apply

Now we apply it through these tasks:

1. Create algorithms for total fare, change from payment, and class average.
2. Write sample test data.
3. Revise one algorithm after tracing.

Challenge Task

Select one real-life numerical problem and write IPO, ordered steps, sample test data, trace, and expected output.

How Our Work Will Be Checked

Criteria - 20 points total

- Correctness (5): Output or answer follows the required concept and instructions.
- Completeness (4): All required parts are present.
- Logic and sequence (4): Steps, code, diagram, or pseudocode follow a clear order.
- Explanation (3): Student can explain what the work does and why it is correct.
- Neatness/readability (2): Work is easy to read, label, and check.
- Effort and revision (2): Student improves the work after feedback.

Let's Reflect

Let's reflect:

- What concept from this lesson is clearest to you now?
- What mistake did you notice or correct during practice?
- Which activity helped you understand the lesson best?
- How can this skill help you design or write a Java program?
- What do you still need to practice before the next lesson?

How We Show Learning

Evidence we will use to check learning:

- Algorithm solves selected problem.
- Sample test proves logic.
- Trace matches expected output.
- Completed guided-practice work with visible corrections or annotations.
- A short explanation connecting the output to the lesson target.

Student Activities

Problem-to-Algorithm Challenge. Trace Table. Logic Check. Rewrite Challenge. Peer Debug. Teacher Quick Check. Create algorithms for total fare, change from payment, and class average. Write sample test data. Revise one algorithm after tracing.

Resources / References

JDK, IDE, projector or screenshots, printed worksheets, notebook, sample code snippets, and teacher-created answer guides.